SERVER SCRIPT

RETURN_UNUSED_MATERIAL

Script Type: API

Event Frequency: All

DocType Event: Before Insert

API Method: return_unused_material

Script

```
# SCRIPT: RETURN UNUSED MATERIAL & STOP WO (Updates 'Returned Qty')
# METHOD NAME: return_unused_material

def return_unused_material():
    try:
        # 1. FETCH WORK ORDER
        wo_id = frappe.form_dict.get("work_order")
        if not wo_id:
            frappe.throw("Work Order ID is required")

        wo = frappe.get_doc("Work Order", wo_id)

        # 2. CALCULATE EXCESS
        transferred_qty = frappe.utils.flt(wo.material_transferred_for_manufacturing)
        produced_qty = frappe.utils.flt(wo.produced_qty)
        excess_qty = transferred_qty - produced_qty

        if excess_qty <= 0:
            frappe.msgprint("⚠ No excess material to return.")
            return

        # 3. CREATE RETURN ENTRY
        se = frappe.new_doc("Stock Entry")
        se.stock_entry_type = "Material Transfer for Manufacture"
        se.purpose = "Material Transfer for Manufacture"
        se.work_order = wo.name
        se.company = wo.company
        se.from_warehouse = wo.wip_warehouse
        se.to_warehouse = wo.source_warehouse
        se.use_multi_level_bom = 0

        items_added = False

        # We track returns here to update the WO table later
        return_map = {}

        for item in wo.required_items:
            # Pro-rata calculation: (Required / Total Planned) * Excess
```

```
    # This tells us how much raw material corresponds to the 106 unmade units
    qty_per_unit = frappe.utils.flt(item.required_qty) / frappe.utils.flt(wo.qty)
    return_qty = qty_per_unit * excess_qty

    # Validation: Check WIP Stock (cannot return what you don't have)
    actual_wip = frappe.db.get_value("Bin",
        {"item_code": item.item_code, "warehouse": wo.wip_warehouse},
        "actual_qty") or 0.0

    final_qty = min(return_qty, actual_wip)

    if final_qty > 0:
        se.append("items", {
            "item_code": item.item_code,
            "s_warehouse": se.from_warehouse,
            "t_warehouse": se.to_warehouse,
            "qty": final_qty,
            "transfer_qty": final_qty,
            "uom": item.source_warehouse_address or "Kg",
            "stock_uom": "Kg",
            "basic_rate": item.rate
        })
        items_added = True

        # Save this amount to update the WO row later
        return_map[item.item_code] = final_qty

if items_added:
    se.insert(ignore_permissions=True)
    se.submit()

    # --- THE FIX: UPDATE "RETURNED QTY" COLUMN ---
    # We loop through the Work Order items and update the 'returned_qty' field
    for row in wo.required_items:
        if row.item_code in return_map:
            current_return = frappe.utils.flt(row.returned_qty)
            new_return = current_return + return_map[row.item_code]

            # Direct DB update to the child table
            frappe.db.set_value("Work Order Item", row.name, "returned_qty", new_return)

# 4. STOP THE WORK ORDER
wo.db_set("status", "Stopped")

msg = f"<b>▒ CLOSED!</b><br>"
msg += f"Unused material returned.<br>"
msg += "Work Order status set to <b>Stopped</b>."
```

```
        msg += "Work Order status set to <b>Stopped</b>."

        frappe.response["message"] = {"success": True, "message": msg}

    except Exception as e:
        frappe.log_error(message=str(e), title="WO Return Error")
        frappe.throw(f"Error: {str(e)}")

return_unused_material()
```

Request Limit:	5
Time Window (Seconds):	86400